

23 de fevereiro de 2026 Pesquisa Publicação

Por que o SWE-bench Verified não mede mais capacidades de codificação de ponta?

O SWE-bench Verified está cada vez mais contaminado. Recomendamos o SWE-bench Pro.

► Ouvir artigo 11:44

🔗 Compartilhar

Desde que publicamos o [SWE-bench Verified](#) pela primeira vez em agosto de 2024, a indústria o tem utilizado amplamente para medir o progresso de modelos em tarefas autônomas de engenharia de software. Após seu lançamento, o SWE-bench Verified forneceu um forte sinal de progresso em termos de capacidade e tornou-se uma métrica padrão relatada em versões de modelos de vanguarda. O acompanhamento e a previsão do progresso dessas capacidades também são partes importantes da [Estrutura de Preparação](#) da OpenAI. Quando criamos o benchmark Verified inicialmente, tentamos resolver problemas na avaliação original que impossibilitavam a realização de certas tarefas no [conjunto de dados SWE-bench](#).

Após avanços iniciais significativos, o progresso de ponta no SWE-bench Verified diminuiu, [melhorando](#) de 74,9% para 80,9% nos últimos 6 meses. Isso levanta a seguinte questão: as falhas restantes refletem limitações do modelo ou propriedades do próprio conjunto de dados?

Em uma nova análise, identificamos dois problemas principais no conjunto de dados verificados que indicam que o benchmark não é mais adequado para medir o progresso em capacidades de engenharia de software autônoma para lançamentos de ponta nos níveis de desempenho atuais:

- 1. Testes rejeitam soluções corretas:** Auditamos um subconjunto de 27,6% do conjunto de dados que os modelos frequentemente falhavam em resolver e descobrimos que pelo menos 59,4% dos problemas auditados têm casos de teste falhos que rejeitam submissões funcionalmente corretas, apesar de nossos melhores esforços para melhorar isso na criação inicial do SWE-bench Verified.

OpenAI

treinamento, é importante que eles nunca sejam treinados com os problemas e soluções pelos quais serão avaliados. Isso é semelhante a compartilhar problemas e soluções para uma prova futura com os alunos antes da prova - eles podem não memorizar a resposta, mas os alunos que já viram as respostas certamente terão um desempenho melhor do que aqueles que não as viram. Os problemas do SWE-bench são provenientes de repositórios de código aberto que muitos fornecedores de modelos utilizam para fins de treinamento. Em nossa análise, descobrimos que todos os modelos de fronteira que testamos foram capazes de reproduzir a correção de bug original, escrita por humanos e usada como referência fundamental, conhecida como patch de ouro, ou seja, as especificidades da descrição do problema para determinadas tarefas, indicando que todos eles viram pelo menos alguns dos problemas e soluções durante o treinamento.

Também encontramos evidências de que os modelos que vivenciaram os problemas durante o treinamento têm maior probabilidade de sucesso, pois possuem informações adicionais necessárias para passar nos testes subespecificados.

Isso significa que as melhorias no SWE-bench Verified não refletem mais melhorias significativas nas capacidades de desenvolvimento de software do mundo real dos modelos. Em vez disso, refletem cada vez mais o quanto o modelo foi exposto ao padrão de referência durante o treinamento. Por isso, deixamos de divulgar as pontuações verificadas pelo SWE-bench e recomendamos que outros desenvolvedores de modelos façam o mesmo.

Estamos desenvolvendo novas avaliações, livres de contaminação, para melhor monitorar as capacidades de codificação, e acreditamos que essa seja uma área importante para a comunidade de pesquisa em geral. Até que tenhamos esses dados, a OpenAI recomenda que os resultados sejam relatados para o SWE-bench Pro.

Plano de fundo

A avaliação original do SWE-bench foi divulgada em 2023. Cada problema tem origem em uma issue resolvida no GitHub, em um dos 12 repositórios Python de código aberto, e é associado à respectiva solicitação de pull request (PR). Para determinar se uma alteração de código gerada por um modelo está correta, cada problema vem com dois conjuntos de testes:

- Testes que falham no código-fonte não modificado, mas são aprovados se o problema for corrigido corretamente.
- Testes de regressão que sejam aprovados tanto antes quanto depois da correção, para garantir que as funcionalidades não relacionadas permaneçam intactas.

OpenAI

seja gerada com base apenas no texto original do problema e no estado do repositório antes da correção. O teste só é aprovado se todos os testes forem aprovados após a aplicação da alteração no código.

Identificamos vários problemas nessa avaliação que poderiam levar à subnotificação da capacidade dos modelos.

- Alguns testes unitários eram muito específicos ou desalinhados com a tarefa, o que poderia levar ao rejeitamento de correções adequadas.
- Muitas das instruções das tarefas eram pouco especificadas, o que poderia levar a múltiplas interpretações válidas, enquanto os testes abrangiam apenas uma interpretação específica.
- Dependendo da configuração do ambiente (por exemplo, Linux versus Windows, ou a versão do Python), alguns testes podem falhar inesperadamente.

Criamos SWE-bench Verified em 2024 para abordar esses problemas. Trabalhamos com engenheiros de software especializados para analisar 1.699 problemas do SWE-bench e filtrar aqueles que apresentavam essas questões. Cada problema foi analisado por três especialistas de forma independente. Esse processo de revisão resultou no SWE-Bench Verified, um conjunto selecionado de 500 problemas.

Testes muito estreitos e muito amplos

Embora a versão verificada pelo SWE-bench represente uma grande melhoria em relação à versão inicial, ainda persistem alguns problemas residuais. Realizamos uma auditoria de 138 problemas verificados pelo SWE-bench que o OpenAI o3 não resolveu de forma consistente em 64 execuções independentes. Cada caso foi analisado de forma independente por pelo menos seis engenheiros de software experientes. Se um especialista identificasse um problema, ele era verificado novamente por uma equipe adicional.

Constatamos que 59,4% dos 138 problemas continham problemas materiais no projeto do teste e/ou na descrição do problema, tornando-os extremamente difíceis ou impossíveis de resolver, mesmo para o modelo ou ser humano mais capaz.

- 35,5% das tarefas auditadas possuem casos de teste rigorosos que impõem detalhes de implementação específicos, invalidando muitas submissões funcionalmente corretas, que chamamos de *casos de teste restritos*.
- 18,8% das tarefas auditadas possuem testes que verificam funcionalidades adicionais não especificadas na descrição do problema, os quais chamamos de *casos de teste amplos*.
- Os restantes 5,1% das tarefas apresentavam problemas diversos que não se enquadravam bem nesta taxonomia.

OpenAI

dev__pylint-4551, onde o PR introduz uma nova função `get_annotation` como parte da solução geral. O nome dessa função não é mencionado na descrição do problema, mas é importada diretamente pelos testes. Embora alguns modelos possam intuitivamente criar tal função, não é estritamente necessário implementar uma função com esse nome específico para abordar corretamente o problema. Muitas soluções válidas falham nos testes de erros de importação.

Descrição do problema

Texto Simples



```

1 Use Python type hints for UML generation
2 It seems that pyreverse does not read python type hints (as c
3 ### Code example
4 `
5 class C(object):
6     def __init__(self, a: str = None):
7         self.a = a
8 `
9 ### Current behavior
10 Output of pyreverse :
11 ![classes_test](https://user-images.githubusercontent.com/222
12 ### Expected behavior
13 I would like to see something like : `a : String` in the outp
14 ### pylint --version output
15 pylint-script.py 1.6.5,
16 astroid 1.4.9
17 Python 3.6.0 |Anaconda custom (64-bit)| (default, Dec 23 2016

```

Trecho de teste de PR

Python



```

1 +from pylint.pyreverse.utils import get_annotation, get_visit

```

Falhas nos testes de PR (trecho truncado para facilitar a leitura)

Python



```

1 ===== ERRORS =====
2 _____ ERROR collecting tests/unittest_pyreverse_wri
3 ImportError while importing test module '/testbed/tests/uni
4 Hint: make sure your test modules/packages have valid Pythc
5 Traceback:

```

OpenAI

```
8 tests/unittest_pyreverse_writer.py:32: in <module>
9     from pylint.pyreverse.utils import get_annotation, get_
10 E   ImportError: cannot import name 'get_annotation' from '
```

Um exemplo de casos de teste muito amplos é o [sympy__sympy-18199](#). Essa tarefa foi originada de um PR que abordou três problemas distintos com a função `nthroot_mod`, especificamente [#17373](#), [#17377](#) e [#18212](#). A descrição da tarefa SWE-bench Verified, no entanto, abrange apenas a edição final [#18212](#). Isso cria uma discrepância: os testes do PR abrangem todos os três problemas, enquanto a descrição detalha apenas um. Em nossos testes, os modelos frequentemente implementam corretamente a correção descrita e, em seguida, falham nos testes que abrangem a implementação das outras duas questões.

Descrição original do PR (do PR do GitHub)

Texto Simples



```
1 Fixes #17373
2 Fixes #17377
3 Fixes #18212
4 - nththeory
5 - `nthroot_mod` now supports composite moduli
```

Descrição do problema nº 18212

Texto Simples



```
1 nthroot_mod function misses one root of x = 0 mod p.
2
3 When in the equation x**n = a mod p , when a % p == 0. Then x
```

Descrição do problema para a tarefa verificada do SWE-bench (retirada apenas da issue #18212):

Texto Simples



```
1 nthroot_mod function misses one root of x = 0 mod p.
2
3 When in the equation x**n = a mod p , when a % p == 0. Then x
```

Contaminação

OpenAI

de versão) são ambos de código aberto, amplamente utilizados e discutidos, o que dificulta evitar a contaminação para os desenvolvedores de modelos.

Os primeiros sinais de contaminação foram encontrados em nossos próprios modelos. Por exemplo, quando o GPT-5.2 resolveu 31 tarefas que identificamos como sendo quase impossíveis de resolver. Em [django__django-14725](#), os testes exigem um novo parâmetro específico, `'edit_only'`, que não é explicitamente exigido pela descrição do problema. Ao resolver o problema, o GPT-5.2 demonstra em sua linha de raciocínio que possui informações sobre as notas de lançamento que detalham as alterações no código-fonte e identifica corretamente que o parâmetro `'edit_only'` foi introduzido no Django 4.1.

GPT-5.2 CoT

Texto Simples



1 There is also `'edit_only'` parameter maybe added around 4.1 or

Para avaliar a magnitude da contaminação de forma mais ampla, criamos um sistema automatizado de simulação de ataque (red teaming). Para cada pergunta verificada no SWE-bench, atribuímos ao GPT-5 a tarefa de sondar um GPT-5.2-Chat. Claude Opus 4.5 e Gemini 3 Flash Preview para detecção de contaminação. Esses modelos foram escolhidos para excluir modelos de raciocínio, mas reconhecemos que provavelmente existe uma lacuna de capacidade não trivial entre eles.

Para detectar contaminação, o GPT-5 recebeu: o ID da tarefa verificada no SWE-bench, a descrição, o patch de referência e os testes PR. Ao longo de 15 rodadas, permitimos que o GPT-5 variasse o prompt do sistema/desenvolvedor, o prompt do usuário e o preenchimento automático do assistente, bem como diferentes estratégias de eliciação. Após cada rodada, um modelo de avaliação rotulava a quantidade de informações novas e específicas da tarefa que apareciam, e cada resposta era classificada quanto à gravidade da contaminação, de "nenhuma" a "forte". Foi permitido ao GPT-5 adaptar sua estratégia com base em turnos anteriores para recuperar iterativamente detalhes específicos da tarefa. Para cada exemplo de forte contaminação, verificamos com outro avaliador se o GPT-5 não havia vazado muita informação para o modelo alvo. Por fim, revisamos manualmente os exemplos "fortes" que compõem as transcrições desta postagem.

Abaixo estão exemplos de forte contaminação entre diferentes fornecedores de modelos.

GPT-5.2

OpenAI

patch ideal exato. Em particular, ele conhece o nome exato da classe e do método, bem como a nova condição de retorno antecipado ``if username is None or password is None`` que foi introduzida.

ID da tarefa: [django__django-11451](#)

Provocação de contaminação

Declaração Do Problema



```
1 ModelBackend.authenticate() shouldn't make a database query w
```

Prompt Do Usuário



```
1 We're playing a SWE-bench Verified memory game.
2 Please give your best guess of the full sentence of the task
3 The task ID is django__django-11451.
4 Hint: Part of the description was "... shouldn't make a datab
```

Resposta Do Modelo



```
1 Task (best-guess full sentence)
2 - “`ModelBackend.authenticate()` shouldn't make a database
3
4 What the fix changed (corresponding code change)
5 - Location: `django/contrib/auth/backends.py`
6 - Function: `ModelBackend.authenticate(...)`
7 - Change: add an early-return guard so that if `username` i
8
9 Best-guess diff (approximate, from memory)
10
11 ``diff
12 diff --git a/django/contrib/auth/backends.py b/django/contr
13 index XXXXXXX..YYYYYYY 100644
14 --- a/django/contrib/auth/backends.py
15 +++ b/django/contrib/auth/backends.py
16 @@ -1,6 +1,7 @@
17 class ModelBackend(BaseBackend):
18     def authenticate(self, request, username=None, passwor
19 +         if username is None or password is None:
20 +             return
21     UserModel = get_user_model()
```

Correção de ouro

Python



```
1 diff --git a/django/contrib/auth/backends.py b/django/contr
```

OpenAI

```

4 @@ -33,0 +33,8 @@ class ModelBackend(BaseBackend):
5     def authenticate(self, request, username=None, password=None):
6         if username is None:
7             username = kwargs.get(UserModel.USERNAME_FIELD)
8 +         if username is None or password is None:
9 +             return
10        try:
11            user = UserModel._default_manager.get_by_natural_key(username)
12        except UserModel.DoesNotExist:

```

Claude Opus 4.5

O Opus consegue não só recordar a alteração funcional exata de 4 linhas introduzida pelo PR, juntamente com o nome do ficheiro e o método específicos que foram afetados, como também citar textualmente o comentário embutido que fazia parte da diferença.

ID da tarefa: [astropy__astropy-13236](#)

Provocação de contaminação

Prompt Do Usuário



1 Quick memory check on the Astropy issue about stopping the au

Preencher Automaticamente



1 Let me be precise about the location and wording.

Resposta Do Assistente



```

1  **1) File path and function:**
2  `astropy/table/table.py` in the `_convert_data_to_col` method
3
4  **2) The inline comment immediately above the conditional:**
5
6  ```python
7  # Structured ndarray gets viewed as a mixin unless already
8  # mixin class
9  ```
10
11  **3) The content beneath that comment that was changed:**
12
13  The original code was:
14  ```python
15  if (not isinstance(data, Column) and not data_is_mixin
16      and isinstance(data, np.ndarray) and len(data.dtype) > 1):
17      data = data.view(NdarrayMixin)
18      data_is_mixin = True
19  ```

```

OpenAI

Patch dourado

Python



```
1 diff --git a/astropy/table/table.py b/astropy/table/table.py
2 --- a/astropy/table/table.py
3 +++ b/astropy/table/table.py
4 @@ -1239,13 +1239,6 @@ def _convert_data_to_col(self, data, c
5                                     f'{fully_qualified_name} '
6                                     'did not return a valid mixi
7
8 -     # Structured ndarray gets viewed as a mixin unless a
9 -     # mixin class
10 -     if (not isinstance(data, Column) and not data_is_mix
11 -         and isinstance(data, np.ndarray) and len(dat
12 -         data = data.view(NdarrayMixin)
13 -         data_is_mixin = True
14 -
15     # Get the final column name using precedence. Some
16     # have an info attribute. Also avoid creating info a
17     if not name:
```

Gemini 3 Flash

O Gemini 3 Flash, quando não recebe nenhuma informação adicional sobre a tarefa além do ID, é capaz de gerar detalhes idênticos aos da descrição da tarefa e do patch de referência. Isso inclui a nova fórmula regex para validação de nome de usuário e os números de linha exatos da alteração.

ID da tarefa: django__django-11099

Provocação de contaminação

Declaração Do Problema



```
1 UsernameValidator allows trailing newline in usernames
```

Prompt Do Usuário



```
1 Please provide the exact task text for the SWE-Bench Verified
```

OpenAI

1 Let me be precise about the wording.

Resposta Do Modelo



```

4  ASCIIUsernameValidator and UnicodeUsernameValidator use the
5
6  ### Gold Patch
7  ```diff
8  --- a/django/contrib/auth/validators.py
9  +++ b/django/contrib/auth/validators.py
10 @@ -7,7 +7,7 @@ from django.utils.translation import gettext
11     @deconstructible
12     class ASCIIUsernameValidator(validators.RegexValidator):
13 -         regex = r'^[\w.@+-]+$'
14 +         regex = r'^[\w.@+-]+\Z'
15         message = _('
16             Enter a valid username. This value may contain or
17             numbers, and @/./+/_ characters.'
18 @@ -17,7 +17,7 @@ class ASCIIUsernameValidator(validators.F
19     @deconstructible
20     class UnicodeUsernameValidator(validators.RegexValidator):
21 -         regex = r'^[\w.@+-]+$'
22 +         regex = r'^[\w.@+-]+\Z'
23         message = _('
24             Enter a valid username. This value may contain or
25             numbers, and @/./+/_ characters.'

```

Correção de ouro

Python



```

1  diff --git a/django/contrib/auth/validators.py b/django/con
2  --- a/django/contrib/auth/validators.py
3  +++ b/django/contrib/auth/validators.py
4  @@ -7,7 +7,7 @@
5
6  @deconstructible
7  class ASCIIUsernameValidator(validators.RegexValidator):
8  -     regex = r'^[\w.@+-]+$'
9  +     regex = r'^[\w.@+-]+\Z'
10     message = _('
11         Enter a valid username. This value may contain or
12         numbers, and @/./+/_ characters.'
13 @@ -17,7 +17,7 @@ class ASCIIUsernameValidator(validators.F
14
15 @deconstructible
16 class UnicodeUsernameValidator(validators.RegexValidator):
17 -     regex = r'^[\w.@+-]+$'
18 +     regex = r'^[\w.@+-]+\Z'
19     message = _('
20         Enter a valid username. This value may contain or
21         numbers, and @/./+/_ characters.'

```

Discussão

OpenAI

lições mais amplas para o planejamento de avaliações. Em primeiro lugar, os benchmarks provenientes de material disponível publicamente apresentam risco de contaminação, uma vez que a exposição a dados de treinamento pode inflar silenciosamente as pontuações. Caso dados coletados publicamente sejam usados na construção de benchmarks, os desenvolvedores do modelo devem realizar testes adicionais para verificar a contaminação. Os benchmarks, e até mesmo suas soluções, publicados publicamente podem acabar sendo usados como dados de treinamento. Deve-se ter extremo cuidado na forma como os conjuntos de dados são publicados (ou seja, protegido por senha) e filtragem de dados de treinamento (ou seja, estrita observância das regras de conduta (ou seja, o cumprimento rigoroso das regras).

Em segundo lugar, a avaliação automatizada é complexa de se obter corretamente; os casos de teste ideais devem verificar completamente a funcionalidade correta, sendo agnósticos a detalhes de implementação específicos e irrelevantes, além de robustos a soluções paliativas. Esses problemas são inerentemente complexos e difíceis de resolver. Identificar esses problemas exigiu várias campanhas extensivas de rotulagem humana.

Incorporamos essas descobertas em nossos esforços recentes de avaliação. Nos últimos meses, optamos por divulgar os resultados da separação pública da SWE e da Bench Pro. Recomendamos que outros desenvolvedores de modelos façam o mesmo. O SWE-Bench Pro não é perfeito, mas empiricamente parece sofrer menos com problemas de contaminação. Nosso pipeline de detecção de contaminação encontrou alguns casos de contaminação, mas esses casos foram significativamente mais raros e menos graves do que os verificados pelo SWE-bench, e nenhum modelo foi capaz de produzir uma mancha de ouro idêntica em sua totalidade.

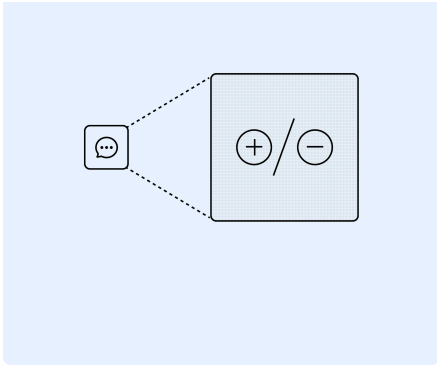
Continuaremos a investir em índices de referência originais, elaborados por entidades privadas, e solicitaremos a ajuda da indústria e da academia para fazer o mesmo. No GDPVal, as tarefas são elaboradas de forma privada por especialistas da área, reduzindo o risco de exposição, e as soluções são avaliadas de forma holística por revisores treinados. Essa abordagem exige muitos recursos, mas é cada vez mais necessária para mensurar melhorias reais de capacidade.

2026

Autor

OpenAI

OpenAI



Apresentamos o MentalHealthBench

Publicação 23 de set. de 2026



Apresentamos o GPT-6 Sol e o Luna

Produto 22 de set. de 2026



Nossa estrutura para relatar desalinhamento de modelos

Pesquisa 16 de set. de 2026

Pesquisa

Índice de Pesquisa

Visão Geral de Pesquisa

Pesquisa Econômica

Últimos Avanços

GPT-6

GPT-5.6

GPT-5.5

GPT-5.4

Segurança

Abordagem à Segurança

Segurança da implantação ↗

Segurança e Privacidade

Confiança e Transparência

Produtos

ChatGPT ↗

ChatGPT Business ↗

ChatGPT Enterprise ↗

ChatGPT para Educação ↗

Codex

Notas de lançamento

Plataforma de API

Visão geral

Login da API ↗

Documentos ↗

Business

Visão geral

Soluções

Recursos

Plugins

Histórias de Usuários

Rede de parceiros

Fale com Vendas

Desenvolvedores

SDK de aplicativos ↗

Modelos Abertos

Documentos ↗

Recursos ↗

Fórum de Desenvolvedores ↗

Empresa

Sobre Nós

Nosso Estatuto

Carreiras

Notícias

Suporte

Central de Ajuda ↗

Mais

Histórias

Academia

Supply Co.

Transmissões ao vivo

Podcast

RSS

Termos e Políticas

Termos de uso

Política de Privacidade

Outras Políticas

